SEAT NO:- 31470002

DR. HOMI BHABHA STATE UNIVERSITY

INSTITUTE OF SCIENCE

DEPARTMENT OF BUSINESS A.I.

ACADEMIC YEAR-2025-26

INTRODUCTION TO ARTIFICIAL INTELLIGENCE JOURNAL

NAME:- YASH BABAN AGRE

CLASS:- SY B.SC.BUSINESS AI

SEAT NO:- 31470002

SUBJECT:- INTRODUCTION TO ARTIFICIAL INTELLIGENCE

SEMESTER:- III

TEACHER SIGN:- _____

DR. HOMI BHABHA STATE UNIVERSITY

INSTITUTE OF SCIENCE

15, MADAM CAMA ROAD, MUMBAI-400 032

Tel. No. (022) 48904545 Email:hbsu.support@digivarsity.com

DEPARTMENT OF BUSINESS A.I. CERTIFICATE

This is to certify that **Mr. YASH BABAN AGRE** Exam Seat No./Roll No. **31470002** B.Sc. Business AI (Semester III) has completed the practical work in the subject Introduction to Artificial Intelligence during the academic Year 2026-27 under the guidance of **PROF. POOJA MALHA** being the partial requirement for the fulfilment of the curriculum of Degree of Bachelor of Science in Business A.I. at The Institute of Science, Dr. Homi Bhabha State University.

Professor In-Charge **Coordinator**

Date: - ……. **College seal**

INDEX

SR.NO.	PRACTICAL TOPIC	DATE	SIGN
1	Python for AI - Setup and Basics		
2	Implementing BFS and DFS		
3	Implementing A* Search		
4	Solving Tic-Tac-Toe using Minimax		
5	Knowledge Representation using Prolog/Python		
6	Bayes Rule and Naive Bayes Classifier		
7	Linear Regression and Decision Tree		
8	K-Means Clustering		
9	Basic Natural Language Processing		
10	Mini AI Project		

Practical 1: Python for AI - Setup and Basics

Aim	To set up the Python AI development environment and practice the core data structures and libraries used throughout the course.
Code	**Procedure: Python & Development Environment Setup** 1. Download Python 3.x from python.org and install it (ensure 'Add Python to PATH' is checked). 2. Verify the installation from the terminal: python --version 3. Install Jupyter Notebook via pip: pip install notebook 4. Launch the environment: jupyter notebook (alternatively, use VS Code with the official Python and Jupyter extensions). 5. Install the libraries: pip install numpy pandas scikit-learn matplotlib 6. Run the verification script, then practise lists, dictionaries, sets, NumPy arrays and Pandas DataFrames. **Python Code:**

```
# Verification script for environment & Python runtime
import sys
import platform

print("Python Version:", platform.python_version())
print("Compiler / Build:", platform.python_compiler())
print("Execution Path:", sys.executable)
print("[STATUS] Python and Jupyter runtime environment successfully initialized.")

# Terminal command executed:
# pip install numpy pandas scikit-learn matplotlib

import numpy as np
import pandas as pd
import sklearn
import matplotlib.pyplot as plt

print("NumPy Version:", np.__version__)
print("Pandas Version:", pd.__version__)
print("scikit-learn Version:", sklearn.__version__)
print("All foundational AI libraries imported successfully!")

# List: product names in stock
products = ["Laptop", "Mouse", "Keyboard", "Monitor"]
products.append("Webcam")   # add an item
products.remove("Mouse")    # remove an item
print("Products:", products)

# Dict: product -> price mapping
prices = {"Laptop": 55000, "Keyboard": 1200, "Monitor": 9500}
prices["Webcam"] = 2200    # add new key-value pair
print("Laptop price:", prices["Laptop"])
print("All prices:", prices)

# Set: unique categories sold this week
categories_mon = {"Electronics", "Accessories", "Electronics"}
categories_tue = {"Accessories", "Furniture"}
print("Unique categories (union):", categories_mon | categories_tue)
print("Common categories (intersection):", categories_mon & categories_tue)

import numpy as np

# Weekly units sold for a product across 7 days
sales_units = np.array([12, 18, 9, 25, 30, 22, 15])
print("Array:", sales_units)
print("Shape:", sales_units.shape)
print("Total units:", sales_units.sum())
print("Average per day:", sales_units.mean())
print("Max day:", sales_units.max())

# Element-wise operation: apply a 10% growth projection
projected = sales_units * 1.10
print("Projected next week:", projected)

# 2D array: units sold for 2 products across 7 days
matrix = np.array([
    [12, 18, 9, 25, 30, 22, 15],
    [8, 14, 11, 19, 21, 17, 13]
])
```

```
print("2D shape:", matrix.shape)
print("Row sums (total per product):", matrix.sum(axis=1))

import pandas as pd

# Series example
prices = pd.Series([55000, 600, 1200, 9500],
                   index=["Laptop", "Mouse", "Keyboard", "Monitor"])
print("Series:\n", prices)

# DataFrame example
data = {
    "Product": ["Laptop", "Mouse", "Keyboard", "Monitor"],
    "Category": ["Electronics", "Accessories", "Accessories", "Electronics"],
    "UnitsSold": [12, 45, 30, 18],
    "UnitPrice": [55000, 600, 1200, 9500]
}
df = pd.DataFrame(data)
print("\nDataFrame:\n", df)

# Basic operations: Revenue calculation and aggregation
df["Revenue"] = df["UnitsSold"] * df["UnitPrice"]
print("\nWith Revenue column:\n", df)
print("\nGrouped by Category:\n", df.groupby("Category")["Revenue"].sum())
```

Output

```
Python Version: 3.12.3
Compiler / Build: GCC 13.3.0
Execution Path: /usr/bin/python3
[STATUS] Python and Jupyter runtime environment successfully initialized.
NumPy Version: 2.4.4
Pandas Version: 3.0.2
scikit-learn Version: 1.8.0
All foundational AI libraries imported successfully!
Products: ['Laptop', 'Keyboard', 'Monitor', 'Webcam']
Laptop price: 55000
All prices: {'Laptop': 55000, 'Keyboard': 1200, 'Monitor': 9500, 'Webcam': 2200}
Unique categories (union): {'Electronics', 'Furniture', 'Accessories'}
Common categories (intersection): {'Accessories'}
Array: [12 18  9 25 30 22 15]
Shape: (7,)
Total units: 131
Average per day: 18.714285714285715
Max day: 30
Projected next week: [13.2 19.8  9.9 27.5 33.  24.2 16.5]
2D shape: (2, 7)
Row sums (total per product): [131 103]
Series:
 Laptop      55000
Mouse         600
Keyboard     1200
Monitor      9500
dtype: int64

DataFrame:
     Product     Category  UnitsSold  UnitPrice
0    Laptop  Electronics         12      55000
1     Mouse  Accessories         45        600
2  Keyboard  Accessories         30       1200
3   Monitor  Electronics         18       9500

With Revenue column:
     Product     Category  UnitsSold  UnitPrice  Revenue
0    Laptop  Electronics         12      55000   660000
1     Mouse  Accessories         45        600    27000
2  Keyboard  Accessories         30       1200    36000
3   Monitor  Electronics         18       9500   171000

Grouped by Category:
 Category
Accessories     63000
Electronics    831000
Name: Revenue, dtype: int64
```

Practical 2: Implementing BFS and DFS

Aim	To represent a graph in Python and implement Breadth-First Search (BFS) and Depth-First Search (DFS) traversal algorithms.
Code	**Procedure: Implementing BFS and DFS** 1. Represent the city map as a graph using an adjacency list (dictionary of lists). 2. Print every node with its neighbours. 3. Implement BFS using a FIFO queue (collections.deque) and a visited set. 4. Implement DFS recursively using a visited set. 5. Implement BFS path finding to obtain the shortest route between two nodes. 6. Run all three functions and compare BFS with DFS. **Python Code:**

```
# Graph representation using an adjacency list (dict of lists)
graph = {
    "Central": ["North", "West"],
    "North": ["Central", "NorthEast", "East"],
    "West": ["Central", "SouthWest"],
    "NorthEast": ["North"],
    "East": ["North", "SouthEast"],
    "SouthWest": ["West"],
    "SouthEast": ["East"]
}

for node, neighbors in graph.items():
    print(f"{node} -> {neighbors}")

from collections import deque

def bfs(graph, start):
    visited = set([start])
    queue = deque([start])
    order = []

    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return order

bfs_order = bfs(graph, "Central")
print("BFS traversal order:", bfs_order)

def dfs(graph, start, visited=None, order=None):
    if visited is None:
        visited = set()
    if order is None:
        order = []

    visited.add(start)
    order.append(start)

    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited, order)
    return order

dfs_order = dfs(graph, "Central")
print("DFS traversal order:", dfs_order)

from collections import deque

def bfs_path(graph, start, goal):
    queue = deque([[start]])
    visited = set([start])

    while queue:
        path = queue.popleft()
        node = path[-1]
        if node == goal:
            return path
```

	`        for neighbor in graph[node]:`
	`            if neighbor not in visited:`
	`                visited.add(neighbor)`
	`                queue.append(path + [neighbor])`
	`    return None`
	
	`path = bfs_path(graph, "Central", "SouthEast")`
	`print("Path from Central to SouthEast:", " -> ".join(path))`

Output	`Central -> ['North', 'West']`
	`North -> ['Central', 'NorthEast', 'East']`
	`West -> ['Central', 'SouthWest']`
	`NorthEast -> ['North']`
	`East -> ['North', 'SouthEast']`
	`SouthWest -> ['West']`
	`SouthEast -> ['East']`
	`BFS traversal order: ['Central', 'North', 'West', 'NorthEast', 'East', 'SouthWest', 'SouthEast']`
	`DFS traversal order: ['Central', 'North', 'NorthEast', 'East', 'SouthEast', 'West', 'SouthWest']`
	`Path from Central to SouthEast: Central -> North -> East -> SouthEast`
	`Algorithmic Comparison of BFS vs. DFS: (table below)`

Algorithmic Comparison of BFS vs. DFS:

Evaluation Aspect	Breadth-First Search (BFS)	Depth-First Search (DFS)
Traversal Style	Level-by-level (breadth-first)	Goes deep along one branch first
Data Structure	FIFO Queue (collections.deque)	LIFO Stack / Function Call Stack (Recursion)
Shortest Path (Unweighted)	Guarantees fewest edge transitions	Does not guarantee shortest path
Primary Use Case	Nearest hub routing, shortest delivery path	Exploring all reachable nodes, cycle detection, topological sorting
Memory Footprint	O(b^d) - stores all nodes at active level	O(b * m) - generally lower memory for deep graphs

Practical 3: Implementing A* Search

Aim	To implement the A* search algorithm using a heuristic function to find the optimal path on a grid.

Code

Procedure: Implementing A* Search

1. Represent the warehouse floor as a 2D grid (0 = open cell, 1 = obstacle).
2. Define the start and goal positions.
3. Implement the Manhattan distance heuristic h(n).
4. Implement A* using a heapq priority queue ordered by f(n) = g(n) + h(n).
5. Run the search and print the optimal path and its cost.
6. Display the grid with the path marked (S = start, G = goal, * = path).

Python Code:

```
# Grid definition: 0 = Open Cell, 1 = Shelving Obstacle
grid = [
    [0, 0, 0, 0, 0],
    [0, 1, 1, 0, 0],
    [0, 0, 1, 0, 0],
    [0, 1, 1, 0, 0],
    [0, 0, 0, 0, 0]
]

start = (0, 0)  # Charging dock
goal = (4, 4)   # Target picking location
print("Start:", start, "| Goal:", goal)
print(f"Grid dimensions: {len(grid)} rows x {len(grid[0])} columns initialized.\n")

def manhattan(a, b):
    # h(n) = |x1 - x2| + |y1 - y2|
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

h_estimate = manhattan(start, goal)
print("Heuristic estimate from start to goal:", h_estimate)

import heapq

def a_star(grid, start, goal):
    rows, cols = len(grid), len(grid[0])
    # Priority Queue stores tuples: (f_score, g_score, current_node, path_taken)
    open_set = [(manhattan(start, goal), 0, start, [start])]
    visited = set()

    while open_set:
        f, g, current, path = heapq.heappop(open_set)

        if current == goal:
            return path, g

        if current in visited:
            continue
        visited.add(current)

        r, c = current
        neighbors = [(r - 1, c), (r + 1, c), (r, c - 1), (r, c + 1)]
        for nr, nc in neighbors:
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 0:
                if (nr, nc) not in visited:
                    new_g = g + 1
                    new_f = new_g + manhattan((nr, nc), goal)
                    heapq.heappush(open_set, (new_f, new_g, (nr, nc), path + [(nr, nc)]))

    return None, float("inf")

path, cost = a_star(grid, start, goal)
print("Optimal path:", path)
print("Path cost:", cost)

def print_grid_with_path(grid, path):
    display = [[str(cell) for cell in row] for row in grid]
    for r, c in path:
        if (r, c) == start:
            display[r][c] = "S"
        elif (r, c) == goal:
            display[r][c] = "G"
        else:
            display[r][c] = "*"
```

```
    for row in display:
        print("  ".join(row))

print(f"Optimal path found with cost = {cost} steps:\n")
print_grid_with_path(grid, path)
```

Output

```
Start: (0, 0) | Goal: (4, 4)
Grid dimensions: 5 rows x 5 columns initialized.

Heuristic estimate from start to goal: 8
Optimal path: [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4), (3, 4), (4, 4)]
Path cost: 8
Optimal path found with cost = 8 steps:

S  *  *  *  *
0  1  1  0  *
0  0  1  0  *
0  1  1  0  *
0  0  0  0  G
```

Practical 4: Solving Tic-Tac-Toe using Minimax

Aim	To build a Tic-Tac-Toe game and implement the Minimax algorithm so the AI never loses.

Code

Procedure: Solving Tic-Tac-Toe using Minimax

1. Store the 3x3 board as a 9-element list.
2. Write helper functions to print the board, check for a winner and detect a draw.
3. Implement Minimax with Alpha-Beta pruning (AI = O is maximising, Human = X is minimising).
4. Write best_move() so the AI picks the highest-scoring move.
5. Simulate a game between a human and the AI and print the board after every move.

Python Code:

```
# 3x3 board stored as a 9-element list
board = ["" for _ in range(9)]

def print_board(b):
    for i in range(0, 9, 3):
        row = [b[i] if b[i] != "" else " " for i in range(i, i+3)]
        print(f" {row[0]} | {row[1]} | {row[2]} ")
        if i < 6:
            print("---+---+---")

def check_winner(b, player):
    win_combos = [
        [0, 1, 2], [3, 4, 5], [6, 7, 8],  # Rows
        [0, 3, 6], [1, 4, 7], [2, 5, 8],  # Columns
        [0, 4, 8], [2, 4, 6]              # Diagonals
    ]
    return any(all(b[i] == player for i in combo) for combo in win_combos)

def is_draw(b):
    return "" not in b and not check_winner(b, "O") and not check_winner(b, "X")

print("Initial Clean Board Layout:")
print_board(board)
print("\n[OPTIMIZATION VERIFIED] Alpha-Beta Pruning integrated.")
print("Minimax evaluates all game-tree branches and prunes redundant subtrees when beta <=
alpha.\n")

def minimax_ab(b, depth, is_maximizing, alpha, beta):
    # Terminal utility evaluation: AI = 'O' (+1), Human = 'X' (-1), Draw = 0
    if check_winner(b, "O"):
        return 1
    if check_winner(b, "X"):
        return -1
    if "" not in b:
        return 0

    if is_maximizing:
        best_score = -float("inf")
        for i in range(9):
            if b[i] == "":
                b[i] = "O"
                score = minimax_ab(b, depth + 1, False, alpha, beta)
                b[i] = ""
                best_score = max(best_score, score)
                alpha = max(alpha, score)
                if beta <= alpha:
                    break  # Beta cutoff
        return best_score
    else:
        best_score = float("inf")
        for i in range(9):
            if b[i] == "":
                b[i] = "X"
                score = minimax_ab(b, depth + 1, True, alpha, beta)
                b[i] = ""
                best_score = min(best_score, score)
                beta = min(beta, score)
                if beta <= alpha:
                    break  # Alpha cutoff
        return best_score

def best_move(b):
    best_score = -float("inf")
```

```
    move = None
    for i in range(9):
        if b[i] == "":
            b[i] = "O"
            score = minimax_ab(b, 0, False, -float("inf"), float("inf"))
            b[i] = ""
            if score > best_score:
                best_score = score
                move = i
    return move

# Demonstration game trace where Human plays X and AI plays O
sample_moves = [4, 2, 7]  # Simulated Human moves: center, corner, edge
sim_board = ["" for _ in range(9)]

print("--- Simulated Game Trace ---")
for h_move in sample_moves:
    sim_board[h_move] = "X"
    print(f"Human plays at {h_move}:")
    if check_winner(sim_board, "X") or is_draw(sim_board):
        break
    ai_pos = best_move(sim_board)
    sim_board[ai_pos] = "O"
    print(f"AI counters at {ai_pos}:")
    print_board(sim_board)
    print("-" * 15)
    if check_winner(sim_board, "O") or is_draw(sim_board):
        break
# Final result of the simulated game
win_combos = [[0,1,2],[3,4,5],[6,7,8],[0,3,6],[1,4,7],[2,5,8],[0,4,8],[2,4,6]]
if check_winner(sim_board, "O"):
    line = next(c for c in win_combos if all(sim_board[i] == "O" for i in c))
    print(f"Result: AI Wins! Line detected across {line}.")
elif check_winner(sim_board, "X"):
    print("Result: Human Wins!")
else:
    print("Result: Draw.")
```

Output

```
Initial Clean Board Layout:
   |   |
---+---+---
   |   |
---+---+---
   |   |

[OPTIMIZATION VERIFIED] Alpha-Beta Pruning integrated.
Minimax evaluates all game-tree branches and prunes redundant subtrees when beta <= alpha.

--- Simulated Game Trace ---
Human plays at 4:
AI counters at 0:
 O |   |
---+---+---
   | X |
---+---+---
   |   |
---------------
Human plays at 2:
AI counters at 6:
 O |   | X
---+---+---
   | X |
---+---+---
 O |   |
---------------
Human plays at 7:
AI counters at 3:
 O |   | X
---+---+---
 O | X |
---+---+---
 O | X |
---------------
Result: AI Wins! Line detected across [0, 3, 6].
```

Practical 5: Knowledge Representation using Prolog/Python

Aim	To represent facts and rules for an organizational hierarchy and perform forward- and backward-chaining logical inference.
Code	**Procedure: Knowledge Representation using Prolog/Python** 1. Store the organizational hierarchy as facts in the form (child, parent). 2. Define the rules is_parent() and is_ancestor(). 3. Implement forward chaining to derive all indirect reporting relationships. 4. Implement backward chaining with a reasoning trace. 5. Query the knowledge base and print the inferences. **Python Code:**

```
# Knowledge base: parent(child, parent) means child reports to parent
facts = [
    ("Aarav", "Priya"),   # Aarav reports to Priya
    ("Priya", "Rohit"),   # Priya reports to Rohit
    ("Sneha", "Priya"),   # Sneha reports to Priya
    ("Rohit", "Kavya")    # Rohit reports to Kavya (CEO)
]

def is_parent(child, parent):
    return (child, parent) in facts

def is_ancestor(person, ancestor, visited=None):
    if visited is None:
        visited = set()
    for child, parent in facts:
        if child == person and parent not in visited:
            if parent == ancestor:
                return True
            visited.add(parent)
            if is_ancestor(parent, ancestor, visited):
                return True
    return False

print("Is Priya direct manager of Aarav?", is_parent("Aarav", "Priya"))
print("Is Rohit direct manager of Aarav?", is_parent("Aarav", "Rohit"))
print("Is Rohit an ancestor of Aarav?", is_ancestor("Aarav", "Rohit"))
print("Is Kavya an ancestor of Aarav?", is_ancestor("Aarav", "Kavya"))

def forward_chain(facts):
    derived = set(facts)
    changed = True
    while changed:
        changed = False
        new_facts = set()
        for (a, b) in derived:
            for (c, d) in derived:
                if b == c and (a, d) not in derived and (a, d) not in new_facts:
                    new_facts.add((a, d))
        if new_facts:
            derived.update(new_facts)
            changed = True
    return derived

all_relationships = forward_chain(facts)
print("All direct + derived reporting relationships:")
for rel in sorted(all_relationships):
    print(f"  {rel[0]} reports up to {rel[1]}")

def backward_chain(query_child, query_ancestor, facts, trace=None):
    if trace is None:
        trace = []
    for (child, parent) in facts:
        if child == query_child:
            trace.append(f"Checking: does {child} report directly to {parent}?")
            if parent == query_ancestor:
                trace.append(f"  Match found: {child} -> {parent}. Goal reached.")
                return True, trace
            else:
                trace.append(f"  Not a direct match. Recursing on {parent} ->
{query_ancestor}")
                result, trace = backward_chain(parent, query_ancestor, facts, trace)
                if result:
                    return True, trace
```

```
    return False, trace

result, trace = backward_chain("Aarav", "Kavya", facts)
print(f"Query Result: Does Aarav report up to Kavya? {result}\n")
print("Reasoning steps (Trace):")
for step in trace:
    print(" ", step)
```

Output

```
Is Priya direct manager of Aarav? True
Is Rohit direct manager of Aarav? False
Is Rohit an ancestor of Aarav? True
Is Kavya an ancestor of Aarav? True
All direct + derived reporting relationships:
  Aarav reports up to Kavya
  Aarav reports up to Priya
  Aarav reports up to Rohit
  Priya reports up to Kavya
  Priya reports up to Rohit
  Rohit reports up to Kavya
  Sneha reports up to Kavya
  Sneha reports up to Priya
  Sneha reports up to Rohit
Query Result: Does Aarav report up to Kavya? True

Reasoning steps (Trace):
  Checking: does Aarav report directly to Priya?
    Not a direct match. Recursing on Priya -> Kavya
  Checking: does Priya report directly to Rohit?
    Not a direct match. Recursing on Rohit -> Kavya
  Checking: does Rohit report directly to Kavya?
    Match found: Rohit -> Kavya. Goal reached.
```

Practical 6: Bayes Rule and Naive Bayes Classifier

Aim	To apply Bayes' theorem mathematically, then train and evaluate a Naive Bayes text classification model using scikit-learn.
Code	**Procedure: Bayes Rule and Naive Bayes Classifier** 1. State the prior probabilities and likelihoods. Bayes Rule: P(Spam | free) = [P(free | Spam) x P(Spam)] / P(free), where P(free) = P(free | Spam) x P(Spam) + P(free | Ham) x P(Ham). 2. Compute P(free) using the Law of Total Probability, then apply Bayes Rule. 3. Create a labelled email dataset and convert the text into a bag-of-words matrix with CountVectorizer. 4. Split the data into training and test sets and train a MultinomialNB model. 5. Predict the class of an unseen email. 6. Evaluate the model using accuracy, confusion matrix and classification report. **Python Code:**

```
# Given prior probabilities
p_spam = 0.4
p_ham = 0.6

# Likelihoods derived from training vocabulary
p_free_given_spam = 0.6
p_free_given_ham = 0.1

# Law of Total Probability: P("free") = P("free"|Spam)*P(Spam) + P("free"|Ham)*P(Ham)
p_free = (p_free_given_spam * p_spam) + (p_free_given_ham * p_ham)

# Bayes' Theorem: P(Spam | "free") = [P("free" | Spam) * P(Spam)] / P("free")
p_spam_given_free = (p_free_given_spam * p_spam) / p_free

print(f"Total probability of word 'free' P(Free): {p_free:.3f}")
print(f"P(Spam | word='free') = {p_spam_given_free:.2f} ({p_spam_given_free * 100:.1f}%)")

import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

data = {
    "Text": [
        "win a free prize now",
        "claim your free lottery winnings",
        "meeting scheduled for tomorrow at 10am",
        "please review the attached report",
        "you have won a cash reward click here",
        "lunch with the team on friday",
        "urgent verify your account to claim prize",
        "project deadline extended to next week",
        "exclusive free offer just for you",
        "can you send me the sales figures"
    ],
    "Label": ["spam", "spam", "ham", "ham", "spam", "ham", "spam", "ham", "spam", "ham"]
}
df = pd.DataFrame(data)
print("Sample Emails Dataset (First 4 rows):\n", df.head(4))

# Vectorize text using bag-of-words representation
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(df["Text"])
y = df["Label"]

print("\nVocabulary Size:", len(vectorizer.get_feature_names_out()))
print("Feature matrix shape:", X.shape)

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
model = MultinomialNB()
model.fit(X_train, y_train)

# Inference on unseen sample email
new_email = ["claim your free reward today"]
new_vec = vectorizer.transform(new_email)
prediction = model.predict(new_vec)
print("Prediction for 'claim your free reward today':", prediction[0])
```

```
# Evaluation
y_pred = model.predict(X_test)
print(f"\nTest Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print("\nConfusion Matrix [labels=['spam', 'ham']]:\n", confusion_matrix(y_test, y_pred,
labels=["spam", "ham"]))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Output

```
Total probability of word 'free' P(Free): 0.300
P(Spam | word='free') = 0.80 (80.0%)
Sample Emails Dataset (First 4 rows):
                                   Text Label
0                    win a free prize now  spam
1        claim your free lottery winnings  spam
2  meeting scheduled for tomorrow at 10am   ham
3       please review the attached report   ham

Vocabulary Size: 48
Feature matrix shape: (10, 48)
Prediction for 'claim your free reward today': spam

Test Accuracy: 1.00

Confusion Matrix [labels=['spam', 'ham']]:
 [[2 0]
 [0 1]]

Classification Report:
              precision    recall  f1-score   support

         ham       1.00      1.00      1.00         1
        spam       1.00      1.00      1.00         2

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3
```

Practical 7: Linear Regression and Decision Tree

Aim	To build and evaluate a Linear Regression model on housing price data and a Decision Tree classifier on credit loan approval data.

Code	**Procedure: Linear Regression and Decision Tree**
	1. Create the housing dataset (HouseSize_sqft, Price_Lakh).
	2. Fit a LinearRegression model and read the slope, intercept and R-squared score.
	3. Predict the price of a 1700 sqft apartment.
	4. Create the loan approval dataset (Income, CreditScore, Employed, LoanApproved).
	5. Split the data and train a DecisionTreeClassifier (max_depth = 3).
	6. Evaluate the accuracy and print the decision rules.
	
	Python Code:
	`import pandas as pd`
	`from sklearn.linear_model import LinearRegression`
	
	`housing = pd.DataFrame({`
	`"HouseSize_sqft": [800, 1000, 1200, 1500, 1800, 2000, 2200, 2500],`
	`"Price_Lakh": [40, 48, 58, 70, 85, 95, 102, 118]`
	`})`
	
	`X = housing[["HouseSize_sqft"]]`
	`y = housing["Price_Lakh"]`
	
	`reg_model = LinearRegression()`
	`reg_model.fit(X, y)`
	
	`intercept = reg_model.intercept_`
	`slope = reg_model.coef_[0]`
	`r_squared = reg_model.score(X, y)`
	
	`print(f"Regression Equation: Price = {slope:.4f} * HouseSize + {intercept:.4f}")`
	`print(f"R² Score: {r_squared:.4f}")`
	
	`# Valuation for a 1700 sqft apartment`
	`pred_1700 = reg_model.predict(pd.DataFrame({"HouseSize_sqft": [1700]}))[0]`
	`print(f"Predicted price for 1700 sqft: {pred_1700:.2f} Lakh")`
	
	`from sklearn.tree import DecisionTreeClassifier, export_text`
	`from sklearn.model_selection import train_test_split`
	`from sklearn.metrics import accuracy_score`
	
	`loan = pd.DataFrame({`
	`"Income": [35000, 60000, 28000, 75000, 42000, 25000, 90000, 32000],`
	`"CreditScore": [650, 720, 600, 750, 680, 580, 780, 610],`
	`"Employed": [1, 1, 0, 1, 1, 0, 1, 0],  # 1 = Yes, 0 = No`
	`"LoanApproved": ["No", "Yes", "No", "Yes", "Yes", "No", "Yes", "No"]`
	`})`
	
	`X_loan = loan[["Income", "CreditScore", "Employed"]]`
	`y_loan = loan["LoanApproved"]`
	
	`X_train, X_test, y_train, y_test = train_test_split(X_loan, y_loan, test_size=0.25,`
	`random_state=42)`
	
	`tree_model = DecisionTreeClassifier(max_depth=3, random_state=42)`
	`tree_model.fit(X_train, y_train)`
	
	`y_pred = tree_model.predict(X_test)`
	`print(f"Decision Tree Test Accuracy: {accuracy_score(y_test, y_pred) * 100:.1f}%")`
	`print("\nDecision Tree Rules:\n", export_text(tree_model, feature_names=["Income",`
	`"CreditScore", "Employed"]))`

Output	`Regression Equation: Price = 0.0458 * HouseSize + 2.6410`
	`R² Score: 0.9989`
	`Predicted price for 1700 sqft: 80.43 Lakh`
	`Decision Tree Test Accuracy: 100.0%`
	
	`Decision Tree Rules:`
	`|--- CreditScore <= 665.00`
	`|   |--- class: No`
	`|--- CreditScore >  665.00`
	`|   |--- class: Yes`

Practical 8: K-Means Clustering

Aim	To apply k-means clustering on a 2D store dataset, determine the optimal number of clusters using the Elbow Method, and visualize customer segments.

Code	**Procedure: K-Means Clustering**
	1. Create the store dataset (DailyFootfall, DailyRevenue_Th).
	2. Standardize the features using StandardScaler (Z-score).
	3. Run KMeans for k = 1 to 7 and record the inertia (Elbow Method).
	4. Choose the optimal k = 3 and fit the final model.
	5. Convert the centroids back to original units using inverse_transform.
	6. Summarize the mean footfall and revenue of each segment.
	
	Python Code:
	`import pandas as pd`
	`from sklearn.preprocessing import StandardScaler`
	
	`stores = pd.DataFrame({`
	`  "StoreID": [f"St{str(i).zfill(2)}" for i in range(1, 11)],`
	`  "DailyFootfall": [850, 900, 200, 180, 500, 520, 870, 210, 510, 890],`
	`  "DailyRevenue_Th": [120, 130, 35, 30, 70, 75, 125, 32, 72, 128]`
	`})`
	
	`X = stores[["DailyFootfall", "DailyRevenue_Th"]]`
	`scaler = StandardScaler()`
	`X_scaled = scaler.fit_transform(X)`
	
	`print("Store Dataset:\n", stores.head(4))`
	`print("\nStandardized Features (First 4 rows):\n", X_scaled[:4])`
	`print("[REASON FOR STANDARDIZATION] Footfall (hundreds) and Revenue (thousands) operate on differing numeric magnitudes.")`
	`print("Without Z-score scaling, footfall would dominate Euclidean distance metrics.\n")`
	
	`from sklearn.cluster import KMeans`
	
	`# Elbow curve calculation across k = 1 to 7`
	`inertias = []`
	`k_range = range(1, 8)`
	`for k in k_range:`
	`  km = KMeans(n_clusters=k, random_state=42, n_init=10)`
	`  km.fit(X_scaled)`
	`  inertias.append(km.inertia_)`
	
	`for k, val in zip(k_range, inertias):`
	`  note = "  <-- Sharp elbow bend detected (Optimal k = 3)" if k == 3 else ""`
	`  print(f"k = {k}: Inertia = {val:.4f}{note}")`
	
	`optimal_k = 3`
	`km_final = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)`
	`stores["Segment"] = km_final.fit_predict(X_scaled)`
	
	`# Inverse transform centroids back to original units`
	`centroids_orig = scaler.inverse_transform(km_final.cluster_centers_)`
	
	`print("Store Segment Assignments:")`
	`print(stores[["StoreID", "DailyFootfall", "DailyRevenue_Th", "Segment"]])`
	
	`# Name each cluster by ranking centroid footfall (low -> high)`
	`order = centroids_orig[:, 0].argsort()`
	`names = {order[0]: "Low Footfall", order[1]: "Mid Performers", order[2]: "Tier-1 Flagship"}`
	
	`print("\nCluster Centroids (Original Metric Units):")`
	`for i, c in enumerate(centroids_orig):`
	`  print(f"Cluster {i} ({names[i]}): Footfall = {c[0]:.1f}, Revenue = {c[1]:.1f}K")`
	
	`print("\nMean Footfall & Revenue by Segment:")`
	`print(stores.groupby("Segment")[["DailyFootfall", "DailyRevenue_Th"]].mean())`

Output	`Store Dataset:`
	`  StoreID  DailyFootfall  DailyRevenue_Th`
	`0  St01  850  120`
	`1  St02  900  130`
	`2  St03  200  35`
	`3  St04  180  30`
	
	`Standardized Features (First 4 rows):`

```
 [[ 1.00915678  0.97540133]
 [ 1.18496806  1.23007531]
 [-1.27638993 -1.18932747]
 [-1.34671444 -1.31666446]]
[REASON FOR STANDARDIZATION] Footfall (hundreds) and Revenue (thousands) operate on
differing numeric magnitudes.
Without Z-score scaling, footfall would dominate Euclidean distance metrics.

k = 1: Inertia = 20.0000
k = 2: Inertia = 3.4571
k = 3: Inertia = 0.0797  <-- Sharp elbow bend detected (Optimal k = 3)
k = 4: Inertia = 0.0372
k = 5: Inertia = 0.0266
k = 6: Inertia = 0.0161
k = 7: Inertia = 0.0074
Store Segment Assignments:
  StoreID  DailyFootfall  DailyRevenue_Th  Segment
0    St01            850              120        1
1    St02            900              130        1
2    St03            200               35        0
3    St04            180               30        0
4    St05            500               70        2
5    St06            520               75        2
6    St07            870              125        1
7    St08            210               32        0
8    St09            510               72        2
9    St10            890              128        1

Cluster Centroids (Original Metric Units):
Cluster 0 (Low Footfall): Footfall = 196.7, Revenue = 32.3K
Cluster 1 (Tier-1 Flagship): Footfall = 877.5, Revenue = 125.8K
Cluster 2 (Mid Performers): Footfall = 510.0, Revenue = 72.3K

Mean Footfall & Revenue by Segment:
         DailyFootfall  DailyRevenue_Th
Segment
0           196.666667        32.333333
1           877.500000       125.750000
2           510.000000        72.333333
```

Practical 9: Basic Natural Language Processing

Aim	To preprocess raw text corpus data using tokenization, stop-word removal, and stemming, then train a sentiment classifier using TF-IDF features.
Code	**Procedure: Basic Natural Language Processing** 1. Create a labelled review dataset (positive / negative). 2. Tokenize the text using lower-casing and a regular expression. 3. Remove stop words and apply a simple suffix-stripping stemmer. 4. Convert the cleaned text into TF-IDF features. 5. Split the data, train a MultinomialNB sentiment classifier and evaluate its accuracy. 6. Predict the sentiment of unseen customer reviews. **Python Code:**

```
import pandas as pd
import re

reviews = pd.DataFrame({
    "Review": [
        "The product quality is excellent and delivery was fast",
        "Terrible experience, the item arrived broken",
        "Really happy with this purchase, highly recommend",
        "Worst customer service ever, very disappointed",
        "Great value for money and works perfectly",
        "The packaging was damaged and support was unhelpful",
        "Excellent build quality, exceeded expectations",
        "Late delivery and poor quality, not satisfied"
    ],
    "Sentiment": ["positive", "negative", "positive", "negative",
                  "positive", "negative", "positive", "negative"]
})

def tokenize(text):
    text = text.lower()
    return re.findall(r"\b[a-z]+\b", text)

reviews["Tokens"] = reviews["Review"].apply(tokenize)
print("Tokenized Reviews (First 3 samples):")
for idx, r in reviews.head(3).iterrows():
    print(f"[{r['Sentiment']}] {r['Tokens']}")

# Built-in English stop words & Porter stemmer demonstration
stop_words = {'the', 'is', 'and', 'was', 'this', 'with', 'for', 'very', 'not'}

# Algorithmic suffix stripper modeling Porter Stemmer behavior
def simple_stem(word):
    for suffix in ['ing', 'ly', 'ed', 'es', 's']:
        if word.endswith(suffix) and len(word) > len(suffix) + 2:
            return word[:-len(suffix)]
    return word

def clean_tokens(tokens):
    filtered = [t for t in tokens if t not in stop_words]
    stemmed = [simple_stem(t) for t in filtered]
    return stemmed

reviews["CleanTokens"] = reviews["Tokens"].apply(clean_tokens)
print("Cleaned & Stemmed Tokens (First 3 rows):")
for idx, r in reviews.head(3).iterrows():
    print(f"Original: {r['Review'][:35]}...")
    print(f"Cleaned : {r['CleanTokens']}\n")

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Use the cleaned (stop-word removed, stemmed) tokens as TF-IDF input
reviews["CleanText"] = reviews["CleanTokens"].apply(lambda toks: " ".join(toks))

tfidf = TfidfVectorizer()
X_tfidf = tfidf.fit_transform(reviews["CleanText"])
y = reviews["Sentiment"]

print("TF-IDF Vocabulary Size:", len(tfidf.get_feature_names_out()))
```

```
print("TF-IDF Matrix Shape:", X_tfidf.shape)

X_train, X_test, y_train, y_test = train_test_split(X_tfidf, y, test_size=0.25,
random_state=42, stratify=y)
clf = MultinomialNB()
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)
print(f"Sentiment Classifier Accuracy: {accuracy_score(y_test, y_pred) * 100:.1f}%")

# Test on unseen test customer reviews
test_cases = [
    "The delivery was quick and the product works great",
    "Terrible customer service and broken item arrived"
]
test_vec = tfidf.transform([" ".join(clean_tokens(tokenize(t))) for t in test_cases])
preds = clf.predict(test_vec)
for rev, p in zip(test_cases, preds):
    print(f"Review: '{rev}' -> Predicted Sentiment: {p.upper()}")
```

Output

```
Tokenized Reviews (First 3 samples):
[positive] ['the', 'product', 'quality', 'is', 'excellent', 'and', 'delivery', 'was',
'fast']
[negative] ['terrible', 'experience', 'the', 'item', 'arrived', 'broken']
[positive] ['really', 'happy', 'with', 'this', 'purchase', 'highly', 'recommend']
Cleaned & Stemmed Tokens (First 3 rows):
Original: The product quality is excellent an...
Cleaned : ['product', 'quality', 'excellent', 'delivery', 'fast']

Original: Terrible experience, the item arriv...
Cleaned : ['terrible', 'experience', 'item', 'arriv', 'broken']

Original: Really happy with this purchase, hi...
Cleaned : ['real', 'happy', 'purchase', 'high', 'recommend']

TF-IDF Vocabulary Size: 35
TF-IDF Matrix Shape: (8, 35)
Sentiment Classifier Accuracy: 50.0%
Review: 'The delivery was quick and the product works great' -> Predicted Sentiment:
POSITIVE
Review: 'Terrible customer service and broken item arrived' -> Predicted Sentiment:
NEGATIVE
```

Practical 10: Mini AI Project

Aim	To independently build, test, and document a complete end-to-end AI application, integrating the rule systems, pattern matching, and inference logic practiced across earlier practicals.

Code	**Procedure: Mini AI Project**
	1. Define the intents and their trigger phrases (knowledge representation).
	2. Define the standardized responses for every intent.
	3. Implement get_response() using substring pattern matching with a fallback reply.
	4. Test the chatbot with in-domain and out-of-domain queries.
	5. Document the project: problem, approach, knowledge base, test analysis, limitations and future work.
	
	Python Code:
	`# Knowledge representation: Intents and Pattern Variations`
	`intents = {`
	`"greeting": ["hello", "hi", "hey there", "good morning"],`
	`"order_status": ["where is my order", "track my order", "order status", "shipment"],`
	`"return_policy": ["how do i return an item", "return policy", "refund process", "refund"],`
	`"store_hours": ["what are your store hours", "when are you open", "closing time"]`
	`}`
	
	`# Standardized Knowledge Base Responses`
	`responses = {`
	`"greeting": "Hello! Welcome to Support. How can I help you today?",`
	`"order_status": "You can track your order using the tracking link sent to your registered email.",`
	`"return_policy": "Items can be returned within 30 days with the original receipt and packaging.",`
	`"store_hours": "Our retail stores are open from 10:00 AM to 9:00 PM, Monday through Sunday."`
	`}`
	`print(f"Initialized Chatbot with {len(intents)} intents and corresponding domain knowledge templates.")`
	
	`def get_response(user_input):`
	`user_input_clean = user_input.lower().strip()`
	`for intent, phrases in intents.items():`
	`for phrase in phrases:`
	`if phrase in user_input_clean:`
	`return responses[intent]`
	`return "I'm sorry, I didn't understand that. Could you please rephrase your request?"`
	
	`test_inputs = [`
	`"hi there",`
	`"where is my order",`
	`"what's your return policy",`
	`"when are you open on sunday",`
	`"do you sell gaming laptops"  # Out-of-domain query`
	`]`
	
	`for inp in test_inputs:`
	`print(f"User Query : {inp}")`
	`print(f"Bot Output : {get_response(inp)}\n")`

Output	`Initialized Chatbot with 4 intents and corresponding domain knowledge templates.`
	`User Query : hi there`
	`Bot Output : Hello! Welcome to Support. How can I help you today?`
	
	`User Query : where is my order`
	`Bot Output : You can track your order using the tracking link sent to your registered email.`
	
	`User Query : what's your return policy`
	`Bot Output : Items can be returned within 30 days with the original receipt and packaging.`
	
	`User Query : when are you open on sunday`
	`Bot Output : Our retail stores are open from 10:00 AM to 9:00 PM, Monday through Sunday.`
	
	`User Query : do you sell gaming laptops`
	`Bot Output : I'm sorry, I didn't understand that. Could you please rephrase your request?`
	`Formal Project Documentation and Technical Review: (table below)`

Formal Project Documentation and Technical Review:

Report Section	Technical Details & Project Documentation
1. Problem Statement	Customer service desks face high repetitive ticket volumes regarding basic store logistics and tracking. This mini-project automates tier-1 instant responses without human agent intervention.
2. Architecture & Approach	Rule-based substring pattern matching was selected over complex ML due to deterministic accuracy requirements, zero training latency, and complete safety against conversational hallucinations.
3. Knowledge Base Structure	Organized into a dual dictionary mapping intent classes to synonym triggers and verified corporate answers.
4. Test Suite Analysis	Successfully classified in-domain queries with 100% precision. Safely triggered graceful fallback responses for out-of-scope inquiries.
5. Architectural Limitations	Vulnerable to typos (e.g. 'ordr statu'), complex multi-intent sentences, and severe paraphrasing that misses exact keyword stems.
6. Future Enhancements	Integrate Practical 9's TF-IDF vectorizer and Multinomial Naive Bayes intent classifier, supplemented with cosine semantic similarity and Levenshtein distance fuzzy matching.